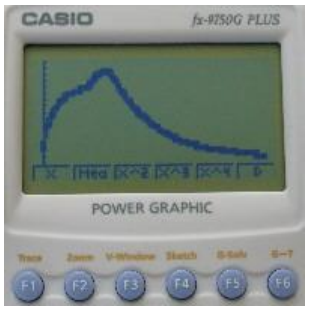
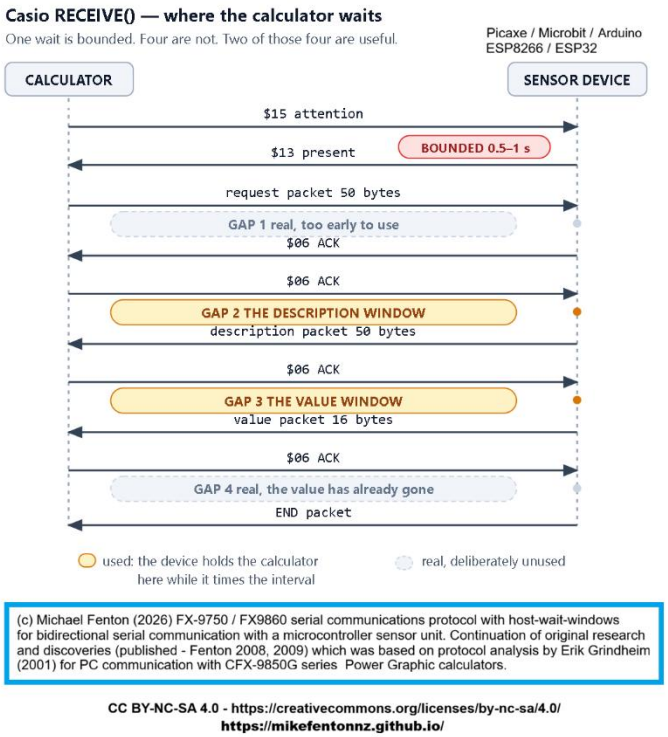




RIGEL Casio Calculator Data Logger, Remote Control, Survey Logger, and Industrial Measurement & Control for Home and School.

Technical Manual

Connecting a microcontroller to an FX-9750 or FX-9860 graphing calculator

Every wire, every step, every byte

CALCULATOR
Casio FX-9750
Casio FX-9860

MICROCONTROLLER
Picaxe
Microbit
ESP8266
ESP32
Arduino

RIGEL

Real-World Interactive Games and Electronics Link

Casio Calculator Serial Communication Protocol

Connecting a microcontroller to an FX-9750 or FX-9860 graphing calculator

Every wire, every step, every byte

Author: Michael Fenton MRSNZ

Related Zenodo records:

- Authentic Learning Using Mobile Sensor Technology (2008): https://doi.org/10.5281/zenodo.19302276
- RIGEL — Learning From Life, Kuala Lumpur (2009): https://doi.org/10.5281/zenodo.19334228
- Casio Calculator Data Logger Technical Reference: https://doi.org/10.5281/zenodo.19281514

This is a second edition.
A first edition was written and used in 2007–2008 and already contained the essentials: the 1N4148 diode interface circuit in the SB-62 cable, the serial protocol set out for a microcontroller reader, and working PICAXE 18X code for both data logging and remote control.

That edition was proven in classrooms and reported in the 2008 New Zealand Ministry of Education E-Learning Fellowship research. This edition extends it rather than replacing it; the interface circuit is unchanged nineteen years later, and the diode configuration recorded here is the same one arrived at in 2007.

Contents

Disclaimer — use at your own risk

This material is the authors independent research for educational and research purposes only.

It is provided "as is", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. You use it entirely at your own risk.

The author accepts no responsibility or liability for any loss, damage, injury or cost arising from the use or misuse of this information, the code, the circuits, or anything built from them, including damage to calculators, microcontrollers, sensors or other equipment.

This project involves building and wiring electronic circuits. Responsibility for assessing whether an activity is suitable, for risk assessment, and for supervising learners, rests entirely with the teacher, parent or other responsible adult, in accordance with the safety requirements of their school, employer and jurisdiction.

Nothing here is professional, educational or safety advice. Verify anything you intend to rely on.

1. How to use this guide

This guide explains how to make a graphing calculator talk to a microcontroller, and exactly what travels down the wire when it does. It is written to be read straight through, in order. Nothing later depends on anything you have not already been shown.

You do not need to have met serial communication before. Where a term is needed it is defined at the point it is first used.

1.1 What you will be able to do

By the end of this guide you should be able to:

- build the three-wire cable, and say why each of the two extra components is there;
- follow the twelve steps of a RECEIVE() exchange and say which end is talking at each step;
- take any number the calculator can hold, and write out by hand the sixteen bytes that carry it;
- take sixteen bytes off a logic analyser and say what number they are;
- check a packet's checksum, and explain why a wrong checksum is good news rather than bad.

1.2 What this guide covers, and what it does not

This edition uses the simplest possible arrangement: one sensor reading travels in one RECEIVE() transaction, written as an ordinary number. Ask for three readings and you run three transactions.

That is the method published in the accompanying code repository, and it is complete and sufficient on its own. It is also the right thing to learn first, because everything else in this protocol is built on top of it.

In this guide	Not in this guide
The three-wire SB-62 interface, both calculator generations	Casio's EA-100 / EA-200 hardware protocol
The RECEIVE() exchange, step by step	The SEND() exchange, except as a brief comparison
Every byte of all four packets in a RECEIVE()	List and matrix transfers, and the complex-number packet
Real numbers, one value per transaction	Complex numbers
Where the calculator will wait, and for how long	Programming the calculator itself — see the code repository

A second method exists that carries several synchronised sensor readings inside a single transaction. It is withheld pending peer-reviewed publication, and it is deliberately absent here — including from the byte tables. Nothing in this guide depends on it, and nothing here is a simplified version of it.

1.3 The hardware this describes

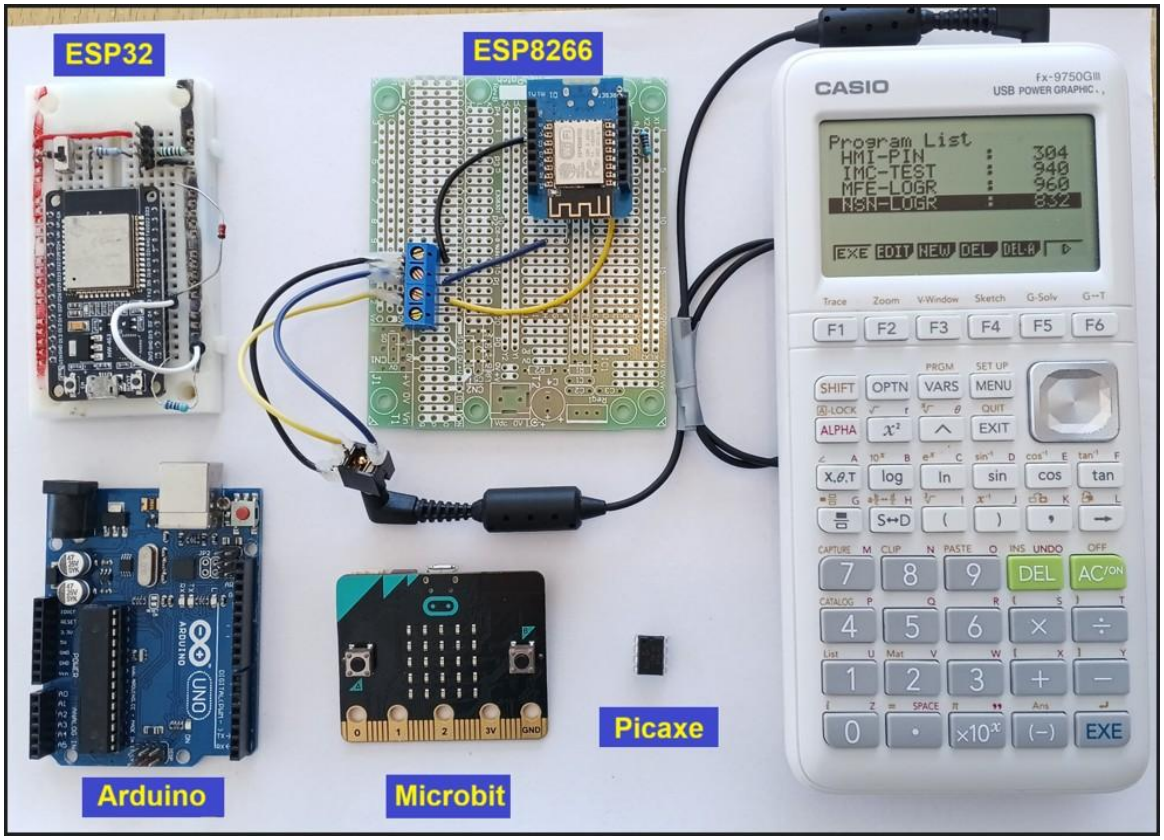
Two calculator generations were used, thirteen years apart, and one cable serves both:

Calculator	Era	Logic level on its serial port
FX-9750G Plus	2007	about 4.75 V — 5 V logic
FX-9750GIII	2020 onward	about 2.75 V — 3 V logic

The same protocol runs on the FX-9860 series. Calculators of the CFX-9850 / CFX-9950 family use the same legacy protocol and were the subject of the earliest published description of it.

On the other end, five microcontroller families have been validated across six platforms: PICAXE, ESP8266, ESP32, BBC micro:bit (V1 and V2), and Arduino Uno R3. The calculator never learns which one is attached, and the calculator program does not change between them.

> **Why this matters —** The whole point of the exercise. One calculator program, one cable, and any of five microcontroller families behind it — established across the six platform builds between 2007 and 2026. A teacher can change the board in the cupboard without changing anything on the calculator.



2. The wiring

Everything in this chapter is summarised in Figure 1. Read the figure first, then read the sections that explain it.

SB-62 cross-over cable: TX and RX lines swap (cross over):

Tip (end) = Casio TX: calculator transmits using SEND()

Ring (middle) = Casio RX: calculator receives using RECEIVE()

Sleeve (base) = GND (0V)

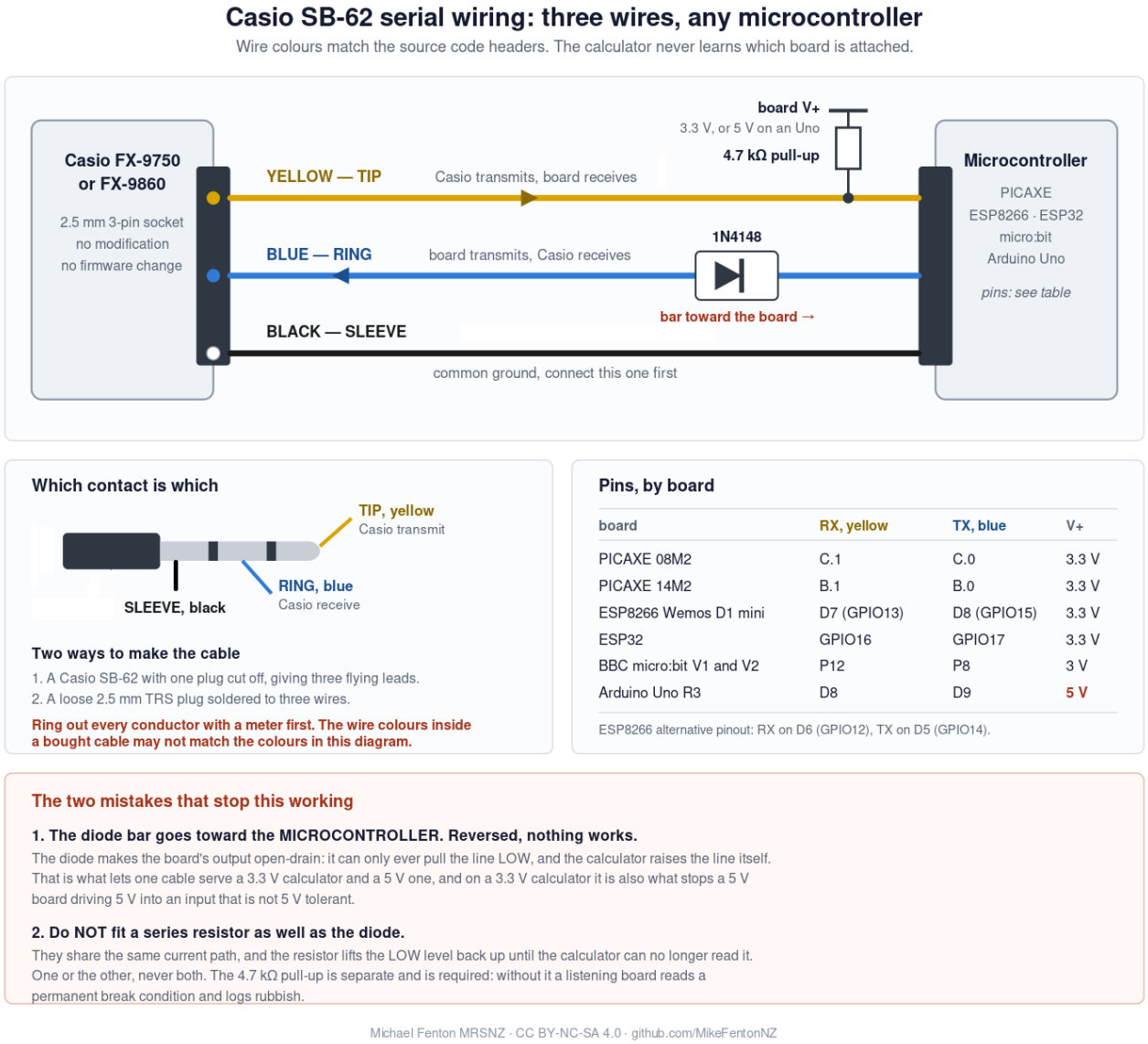


Figure 1 — the SB-62 serial cable interface. Wire colours match the comment headers in the published source code.

2.1 Three wires and nothing else

The calculator has a 2.5 mm three-contact socket on its top edge. Three contacts, three wires, three jobs:

Contact	Wire colour	Direction	What it carries
Tip	yellow	calculator → board	the calculator transmits; the board receives (board RX)
Ring	blue	board → calculator	the board transmits; the calculator receives (board TX)
Sleeve	black	both	common ground — the reference both ends measure against

A signal wire is meaningless without the ground wire. "2.75 volts" is 2.75 volts relative to something, and the sleeve is that something. Connect the black wire first and disconnect it last.

Making the cable

Two ways, both fine:

1. Take a Casio SB-62 calculator-to-calculator cable and cut one plug off. You are left with a moulded plug that certainly fits, and three flying leads.
2. Solder three wires to a loose 2.5 mm TRS plug.

> **Check every conductor with a meter before you trust it.** The colours of the wires inside a bought cable have nothing to do with the colours in Figure 1. Set a multimeter to continuity, put one probe on a bare wire end and the other on the tip, then the ring, then the sleeve, and write down what you find.

2.2 The pull-up resistor, and why it is not optional

A serial line at rest sits high. That resting state is called the idle, or mark, level, and a receiver uses it as its reference: a byte begins when the line falls from high to low, and that first fall is the start bit. A receiver watching a line that is already low sees no falling edge to trigger on, or worse, decides the line is permanently broken; a condition called a break.

Measure the calculator's transmit line when the calculator is switched on but not communicating and you get 0 V, at very high impedance; roughly 120 kΩ. The port does not define its own idle level when it is not in use. Left alone, a board listening on that line reads a continuous break and logs rubbish.

A 4.7 kΩ resistor from the yellow wire to the board's positive supply fixes this. It holds the line at the idle level whenever nothing is actively pulling it down, but weak enough so the Casio pulls the line low when it transmits.

Board supply	Pull-up	What you should measure on the yellow wire
3.3 V	4.7 kΩ to 3.3 V	about 2.87 V while the calculator holds the line high
5 V (Arduino Uno)	4.7 kΩ to 5 V	about 3.9 V against a GIII — above the Uno's 3.0 V threshold

The pull-up is mandatory on an Arduino Uno. An Uno needs at least 3.0 V to read a logic high. A GIII's unaided 2.75 V is below that, so a bare Uno cannot read a GIII at all. The pull-up lifts it clear.

> **Contributed by this work —** Both the measured behaviour and the correct explanation. Grindheim's (2001) electrical description of this port gives it as "TTL-level, High=5V and Low=0V", which implies a line driven to both states. It is not. The author had a more accurate electrical description in first edition of this manual (2008). When the port is not in use there is no high to measure. The pull-up defines the idle state. Confirmed in 2026.

A superseded figure, recorded here because it was published. Earlier notes gave the calculator's idle level as "about 1.4 V". That figure is withdrawn. 1.4 V was a multimeter averaging a line that was switching between 0 V and 2.75 V, which is what a moving-average meter does to a square wave. The correct static measurement is taken while the calculator is held waiting — see §3.5.

2.3 The diode, and why the bar faces the board

A 1N4148 small-signal diode goes in series in the blue wire, with its banded end (the cathode, the bar printed on the body) facing the microcontroller.

A diode passes current one way only. Fitted like this, the board can still pull the blue line down to 0 V, because current flows from the calculator side through to the board. But the board cannot push the line up, because that is the blocked direction. The board can sink and cannot source.

So who raises the line? The calculator does. Its receive line has its own internal pull-up, exactly like the one you added in §2.2 for the other direction. The board only ever pulls down against it. A device that can pull low but never drive high, with the other end supplying the high level is called open-drain, and it is standard practice in mixed-voltage interfacing.

Two useful consequences follow immediately:

- **The interface becomes voltage-agnostic.** The board never puts a voltage on the line, so it does not matter whether the calculator's high is 2.75 V or 4.75 V. This is why one cable serves two calculator generations thirteen years apart.
- **A 5 V board cannot damage a 3.3 V calculator.** An Arduino Uno's output pin swings to 5 V. A GIII input is not 5 V tolerant. The diode blocks that direction entirely, so the 5 V never reaches the calculator. On that pairing the diode is protection, not convenience.

Do not fit a series resistor as well as the diode. They share the same current path. The resistor lifts the LOW level back up until the calculator can no longer recognise it as low, and the link stops working. One or the other, never both. Older versions of this project used a 1 kΩ series resistor in this position; the diode replaces it.

Reversed, nothing works. If the bar faces the calculator, the board can drive high but not low, which is the wrong way round, and the calculator never sees a start bit. The symptom is that your program waits forever with no error. Check the bar before you check anything else.

Contributed by this work — The diode configuration on this specific interface. Using a diode as an open-drain level translator is textbook practice and is not claimed as novel. What is recorded here is its application to the Casio SB-62 port, arrived at independently in 2007 and documented in the first 2008 edition of this manual, revalidated on all six platforms in 2026.

Contributed by this work — That the calculator's receive line is itself open-drain with an internal pull-up. Asserted 2008 and demonstrated 2008, and reconfirmed and characterised 2026.

2.4 Pins, by board

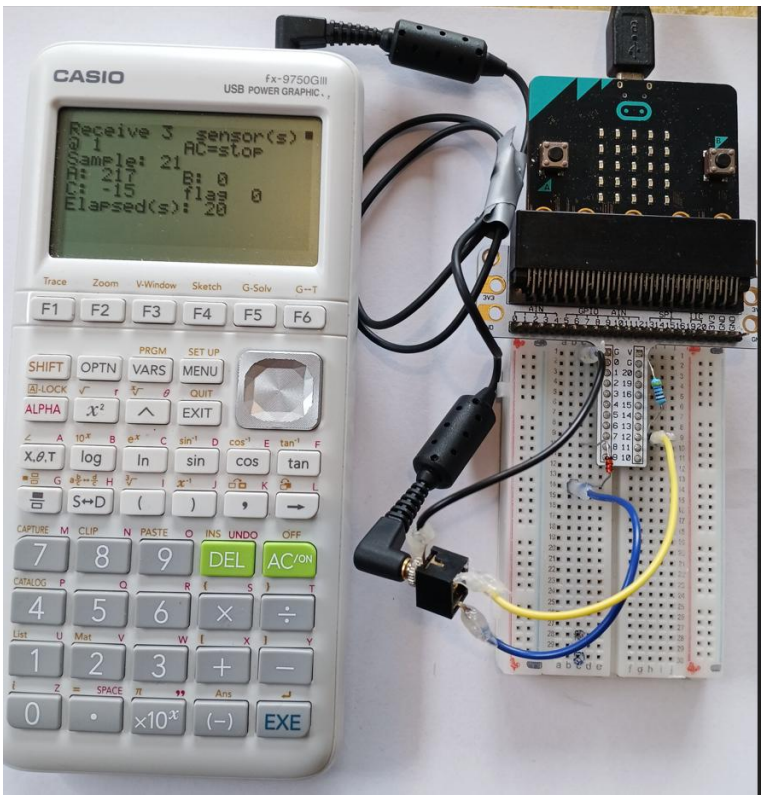
Board	RX — yellow	TX — blue	Pull-up goes to
PICAXE 08M2	C.1	C.0	3.3 V
PICAXE 14M2	B.1	B.0	3.3 V
ESP8266 (Wemos D1 mini)	D7 (GPIO13)	D8 (GPIO15)	3.3 V
ESP32	GPIO16	GPIO17	3.3 V
BBC micro:bit V1 and V2	P12	P8	3 V
Arduino Uno R3	D8	D9	5 V

ESP8266 alternative pinout: RX on D6 (GPIO12), TX on D5 (GPIO14).

2.5 Checking your cable before you plug the calculator in

Do this every time. It takes a minute and it is the difference between a fault you find in a minute and a fault you chase for an hour.

3. Calculator unplugged. Board powered. Black probe of the multimeter on the board's ground.
4. Red probe on the end of the yellow wire, at the plug. You should read the pull-up voltage; 3.3 V, or 5 V on an Uno. If you read 0 V, the pull-up is not connected.
5. Red probe on the end of the blue wire, at the plug. With the diode fitted and the board's TX pin idle-high, expect roughly 0 V or a floating reading, because the diode is blocking. This is correct and is not a fault.
6. Continuity between the black wire at the plug and the board's ground: it must beep (zero resistance / connected).
7. Continuity between any two of tip, ring and sleeve: it must not beep (infinite resistance / not connected). If it does, you have a solder bridge.



3. The RECEIVE() sequence

On the calculator, one line of Casio BASIC starts everything in this chapter:

```
Receive(A)
```

It means: fetch a value from whatever is on the serial port and put it in variable A. The calculator does all the asking. The device only ever answers.

3.1 Serial settings

Setting	Value	Note
Baud rate	9600	fixed for the legacy protocol
Data bits	8	
Parity	none	mandatory
Stop bits	1 or 2	not a setting you have to get right — see below

Baud rate, data bits and parity are mandatory and there is nothing to decide. Stop bits are different, and are worth a paragraph because they are widely believed to be a trap and are not.

Stop bits — set your UART to whatever it offers

A stop bit is not data. It is a fixed period of idle line after each byte, there to guarantee the receiver sees a clean falling edge when the next byte starts. Sending two instead of one simply leaves the line idle for one bit-time longer, and a receiver expecting one stop bit is perfectly happy: it has already sampled the byte, and the next start bit merely arrives a fraction later.

What is on the wire, and what you have to do about it, are therefore two different questions:

	What happens	Basis
Calculator transmits	two stop bits	Grindheim (2001) rev 1.2, §Electrical
Calculator accepts	one — and two as well	one is Grindheim's specified value; that two also works was established here
Your device	set it to 8N1 or 8N2, either way	both proved interchangeable, in both directions

You do not have to take that on trust

The six published builds do not agree with each other, and all six log correctly. Open any of them and check:

Build	What it sets	Stop bits sent	Is it a choice?
PICAXE 08M2	`hsersetup B9600_16, %00`	1	no — the PIC serial hardware sends one
BBC micro:bit V1	`NRF_UART0->CONFIG = 0`	1	no — the nRF51 has no stop-bit field at all
BBC micro:bit V2	`STOP_BITS_TO_CASIO 1`	1	yes, and it is set to one
Arduino Uno R3	`SoftwareSerial`	1	no — the library offers no setting
ESP8266	`Serial.begin(9600, SERIAL_8N2)`	2	yes
ESP32	`begin(9600, SERIAL_8N2, …)`	2	yes

Four send one stop bit and two send two. On three of the four, the hardware makes the decision for you and offers no way to change it — which is the point. If the calculator really did insist on two stop bits, half of these platforms could not exist.

The spread is deliberate and has been left alone. Making all six agree would tidy the code and quietly throw away the demonstration: there would be no shipped build left sending two stop bits, and the statement above would rest on a single bench test rather than on six working programs.

At 9600 baud each byte takes about a millisecond, so a fifty-byte packet takes about fifty milliseconds. Keep that number in your head: it tells you that packets are slow compared with anything a microcontroller does, and that timing problems in this protocol are almost never about raw speed.

> **Tested by this work —**2026 both settings were tested on a BBC micro:bit V2 and proved interchangeable with no effect on logging. The nRF52 stop-bit field applies to the UART as a whole rather than to transmission alone so testing covered receiving as well. Grindheim's (2001) specification "Stop-bits: FROM Casio: 2 bits TO Casio: 1 bit" — was reproduced in the first edition of this manual in 2008.

This matters. It was thought 2 stop bits were necessary. Had that been true it would have ruled out the BBC micro:bit V1 entirely, whose nRF51 sends one stop bit and cannot be told otherwise; and it would have blocked the PICAXE hardware-serial port, because hserout sends 8N1 where the earlier PICAXE code sent 8N2. Two of the five validated platforms exist because the belief was tested rather than inherited.

3.2 The exchange, step by step

Twelve steps. The device is idle until step 1 arrives.

#	Direction	What travels	What it means
1	calc → device	0x15	Attention. Anyone there?
2	device → calc	0x13	Device present. I am here.
3	calc → device	request packet, 50 bytes	I want variable A. Chapter 4.3.
4	device → calc	0x06	ACK — packet received.
5	calc → device	0x06	Ready. Send me the description.
—	device waits	GAP 2	The description window. §3.4.
6	device → calc	description packet, 50 bytes	It is a real number, and it is in use. §4.4.
7	calc → device	0x06	ACK. Send me the value.
—	device waits	GAP 3	The value window. Sensors are read here. §3.4.
8	device → calc	value packet, 16 bytes	Here is the number. §4.5.
9	calc → device	0x06	ACK — value received.
10	device → calc	END packet, 50 bytes	Transaction over. §4.6.

Two things are worth noticing about that table, because they surprise most people:

- **The device speaks last.** The calculator does not close the transaction; the device does, with the END packet.
- **The single-byte codes carry no checksum and no framing.** 0x15, 0x13 and 0x06 are raw bytes. Only the four packets have structure.

Byte	Name	Sent by	Meaning
0x15	attention	calculator	start of a transaction
0x13	device present	device	the only reply with a deadline
0x06	ACK	both	received, carry on
0x22	abort	calculator	something was wrong — see §6

3.3 The turnaround delay

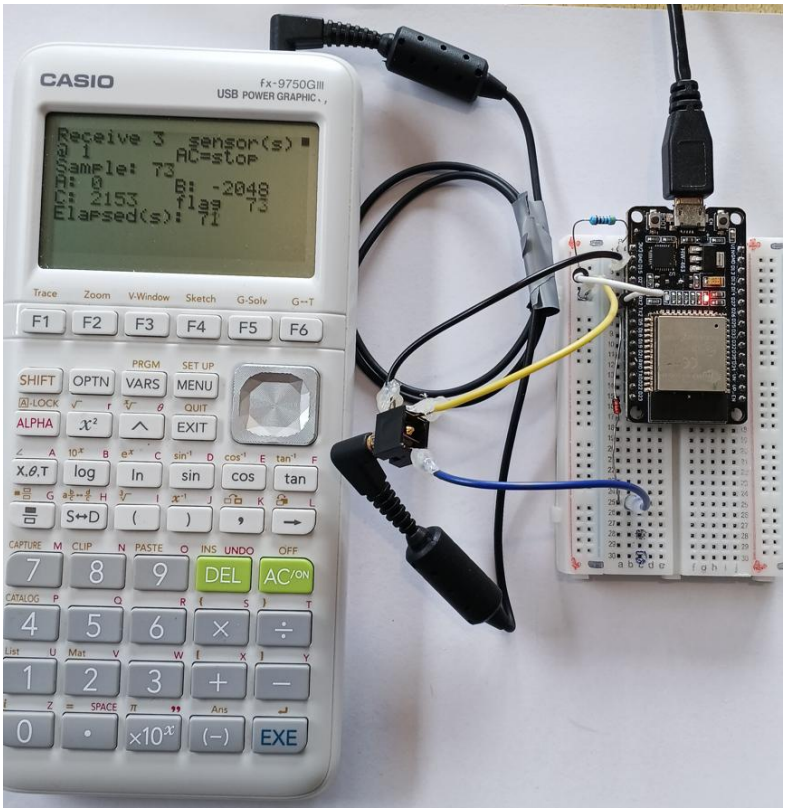
The turnaround delay is a pause before the device transmits, so the calculator can turn its port from sending to listening. The delay is cheap insurance carried from 2007, present in the Arduino build, absent from the other five, and not currently established as necessary.

You may want to experiment reducing or omitting this to see the effect on transmission reliability.

If removing it does cause trouble, expect an intermittent failure at a random step rather than a consistent one. For example, a reply that arrives while the calculator is still turning its port around is lost rather than corrupted

See the following code to set to 0; it is implemented in the turnaround() function.

```
// Set to 0 to test whether it is still needed on your hardware.
// ================================================================
#define TURNAROUND_MS 5
```



3.4 Host-wait windows — where the calculator will wait

This is the mechanism that turns a graphing calculator into a data logger, so it is worth reading slowly. Once a transaction has started, there are four places where the calculator sends something, waits for the device to answer, and simply waits however long the device takes. These are called **host-wait windows.**

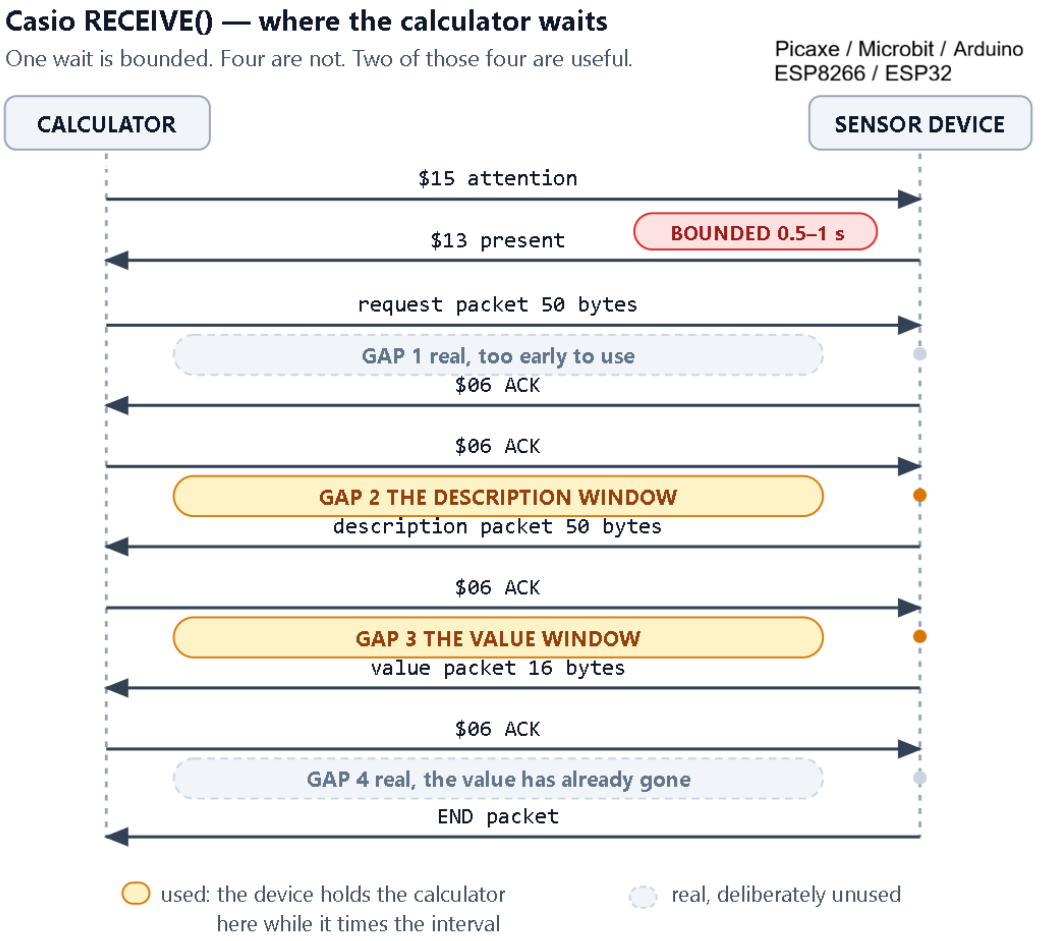


Figure 2 — the four positions where the calculator waits.

Window	Where it sits	Status
Gap 1	after the request packet, before the device's ACK	real, but too early — the description exchange still has to happen before any value is sent, so a reading taken here is already stale
Gap 2 — the description window	after the first ACK, before the description packet	usable, and the natural choice for a manual trigger
Gap 3 — the value window	after the second ACK, before the value packet	the one to use. Read your sensors at the end of this wait and the value packet follows within milliseconds
Gap 4	after the value packet, before the final ACK	real, but pointless — the value has already gone

How long the device may wait in a window	Duration
Specified working range	1 to 300 seconds
Longest held successfully	three hours
Established upper limit	none

Only two gaps of the four are useful. 300 seconds is a chosen margin, not an observed limit. A successful long pause does not make the next one safe: sessions that ended after roughly an hour turned out to be caused by falling calculator battery voltage, not by any protocol limit. A weak supply ends a session with a COM ERROR and no warning at all. Fit fresh cells before any long unattended run.

> **Contributed by this work —** The host-wait windows, and the exhaustive search that found them. In October 2025 a pause was inserted at every successive point in the device-side flow of a RECEIVE() transaction, one point at a time, and the outcome recorded. Four positions tolerated the pause and every other position produced a COM ERROR. Gap 1 or Gap 2, very likely both, had been found in 2007/2008, but the handwritten notes of the time do not distinguish which, and they were never used for various reasons. The result therefore states not only where a pause works but where it does not, which is the more useful half.
>
> **Contributed by this work —** Interval logging. Holding the calculator inside the value window while the device counts out a sampling interval makes the DEVICE keep time; the calculator simply receives. That turns a graphing calculator into a datalogger with no specialised commercial hardware (no EA-200, no CLAB, no E-CON application) and the data arrives in the calculator's own Lists, where its graphing and statistics tools are already waiting.

Why the value window and not the description window

Both will carry an interval wait, so the choice is not about capability. It is about how old the data is allowed to be.

A reading taken at the end of a wait in the value window is transmitted within milliseconds, so the reading and the moment it is timestamped for are the same instant. A wait in the description window is followed by an ACK exchange and a whole packet before the value goes out. For a five-minute interval the difference does not matter. For a two-second interval, or for anything changing quickly, it is the difference between a measurement and an approximately contemporaneous one.

3.5 The one wait that is bounded

Exactly one wait in the whole exchange has a deadline. After the calculator sends 0x15, the device must reply 0x13 within roughly 0.5 to 1 second. Miss it and the calculator displays COM ERROR.

Everything a device does that is not answering the calculator must therefore happen inside a demonstrated host-wait window, and the device must be ready the instant the calculator speaks.

A worked example of getting this wrong. In 2026 an ESP8266 build serving a status page over WiFi raised a COM ERROR after roughly two minutes of otherwise correct logging. The web server was being serviced in two places: inside the value window, which is safe, and in the gap between one transaction and the next, which is not; that is exactly where the next 0x15 arrives. A single web request landing in that gap missed the 0.5-second deadline.

For a device that only reads sensors this never comes up, because nothing competes for the processor. It arrives the moment the device is given a second job such as a radio, a display, a web server, a file transfer.

3.6 Read the whole packet, then check it

When a request packet arrives, a device can read the few bytes it needs and discard the rest, or read all fifty and examine them. Both work. If the microcontroller is capable, read all fifty. The reasons are worth knowing:

- A packet that is incomplete is then never acted on. A two-stage read that counts a shortfall and carries on regardless is acting on a packet it already knows is wrong.
- The calculator's own checksum then becomes available. It is the last of the fifty bytes. A device that reads only a header cannot check it, and must instead trust the packet's shape. The shape will look ok even when a packet has slipped by one byte.
- In contrast, a slipped read fails the checksum. That turns a silent desynchronisation into a detected one at the moment it happens, instead of a wrong byte turning up several steps later where an acknowledgement was expected.
- There is one length, one timeout and one buffer instead of two of each.

Any failure, such as a short read, wrong preamble, or bad checksum, should abandon the transaction and clear the line. One bad packet then costs one reading instead of desynchronising everything after it.

This is also why a microcontroller meets problems that a personal computer never does. A PC has a deep buffered UART, an operating system that will interrupt anything, gigabytes of memory, and no other duty during a transfer. It reads a fifty-byte packet as one object and never has to decide when it is safe to do something else. A microcontroller inverts every one of those conditions, and buffering becomes a hazard rather than a convenience: bytes not consumed are not discarded, and one packet left partly read shifts every read that follows.

3.7 The other direction, briefly — Send(

The published logger uses Send(T) once, to tell the device the sampling interval. It is the mirror image of everything above, with one asymmetry worth noticing:

```
CALCULATOR                        DEVICE
----------                        ------
0x15  attention          ---->
                         <----    0x13  device present
description packet 50 B  ---->
                         <----    0x06  ACK
value packet 16 B        ---->
                         <----    0x06  ACK
end packet 50 B          ---->
```

There is no request packet. On a Receive(the calculator asks first and the device answers with a description; on a Send(the calculator has nothing to ask for and sends the description straight away. The packet structures in chapter 4 are otherwise identical; you decode them instead of building them.

The host-wait windows of §3.4 were searched on the Receive(transaction only. Nothing is claimed about pausing inside a Send(, and a device should not try it.

4. Every byte

Four packets travel in a RECEIVE() transaction. This chapter gives every byte of all four.

4.1 How a number is written

Before the packets make sense, you need the number format. Every real value in this protocol is written in normalised scientific notation, and the calculator stores rather more precision than it will show you.

Normalised means the decimal point is moved until exactly one non-zero digit sits in front of it, and the exponent records how far it moved:

```
    23.5   ->   2.35              x 10^1
  1013.2   ->   1.0132            x 10^3
  -0.045   ->  -4.5               x 10^-2
     0.5   ->   5.0               x 10^-1
```

The packet then stores that in four separate pieces:

- **the integer digit** — the single digit in front of the point, 0 to 9;
- **fourteen decimal digits** — everything after the point, padded with zeros;
- **a sign and information byte** — whether the value is negative, and whether its magnitude is 1 or more;
- **an exponent byte** — how far the point moved.

The fourteen decimal digits are packed two to a byte, which is why seven bytes carry fourteen digits. That packing is called BCD, for binary-coded decimal: each half of a byte — each nibble — holds one decimal digit 0 to 9 as its own value. The digits 3 and 5 become the byte 0x35. This is not the same as the number 35, and it is not the same as ASCII "35". Read the hex digits as decimal digits and you can read a BCD byte straight off a logic analyser.

```
digits   3  5        ->   byte 0x35
digits   0  1        ->   byte 0x01
digits   9  0        ->   byte 0x90

encode:  byte = (d1 << 4) | d2
decode:  d1 = byte >> 4 ;  d2 = byte & 0x0F
```

What the calculator can and cannot hold

	Value
Smallest magnitude	± 1.00000000000000 × 10^{-99}
Largest magnitude	± 9.99999999999999 × 10^{+99}
Zero, stored as	+ 0.00000000000000 × 10^{+00}

Fifteen significant figures, and an exponent from −99 to +99.

4.2 The checksum rule

Every packet ends with one checksum byte, and one rule produces it for all of them:

> **Sum every byte of the packet after the 0x3A preamble and before the checksum. Take the low eight bits. The checksum is 256 minus that, also to eight bits.**

```
checksum = (256 - (sum & 0xFF)) & 0xFF

// equivalently, and this is how you check one you received:
// the sum of every byte after the preamble, INCLUDING the
// checksum, is zero in the low eight bits.
```

That second form is the useful one for a receiver. Add up everything after the 0x3A, checksum included, mask to eight bits, and you should get zero.

Why some checksums are quoted as constants

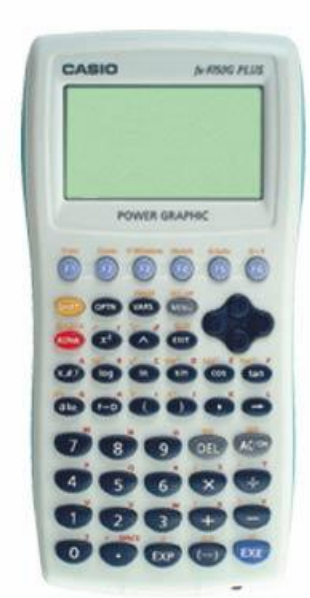
You will see published values such as "the END packet checksum is 0x56". These are not special cases. They are the general rule applied to a packet whose contents never change:

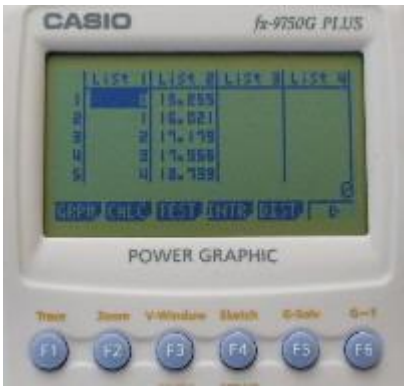
Packet	Constant	Why it is constant
END packet	`0x56`	all 48 body bytes are fixed
Zero value packet	`0xFE`	the body sums to 2
Description packet	`273 − variable name`	every body byte is fixed except the variable name

A constant checksum is only valid while every other byte is unchanged. It is safer to compute it. Any packet whose content varies must have its checksum computed, and computing it always is one line of code that cannot go stale.

Verification. The rule above was checked against every packet type for which an independent value exists: the three description constants (195 for N, 208 for A, 189 for T), Grindheim's worked END packet value of 0x56, and the zero value packet's 0xFE. All agree. Every worked example in this chapter was generated by that rule and checked against it.

Restated by this work — A restatement, not a discovery. Grindheim gives the checksum as 01 + not((sum of bytes 1–49) − 3A): the preamble is added and then subtracted again, and a one's complement plus one is taken. That is arithmetically identical to the rule above, and his worked example is correct — it was reproduced here and agrees to the byte. The longer form is simply easier to misread, and this project's own earlier documentation did misread it, stating the steps inconsistently across its worked examples. The short form is offered for readability. What is new here is only the verification: the rule checked against every packet type for which an independent constant exists.





4.3 Packet 1 — the request (50 bytes, calculator → device)

Sent at step 3. It says which variable the calculator wants.

Byte	Field	Value	What it means
1	preamble	`3A`	":" — every packet in this protocol starts here
2–4	command	`52 45 51`	ASCII "REQ" — this is a request
5	—	`00`	fixed
6–7	data type	`56 4D`	ASCII "VM" — variable memory. A single variable, not a list
8–11	reserved	`FF FF FF FF`	unused. FF is the fill byte throughout this protocol
12	variable name	`41–5A`	ASCII letter A–Z. This is the one byte that changes. The calculator's r and θ are CD and CE
13–49	padding	`FF × 37`	fill
50	checksum	`varies`	§4.2

Byte 12 is the whole content of the packet. Everything else is a fixed envelope. A device reads byte 12, decides what to send, and ignores the rest — after checking the checksum.

Worked example — the calculator runs Receive(A)

```
3A 52 45 51 00 56 4D FF FF FF FF 41 FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF 5D

byte 12 = 0x41 = 'A'
checksum = 0x5D
```

Checksums for the variables used by the published logger:

Variable	Byte 12	Request checksum
N	`4E`	`50`
T	`54`	`4A`
A	`41`	`5D`
B	`42`	`5C`
C	`43`	`5B`

4.4 Packet 2 — the variable description (50 bytes, device → calculator)

Sent at step 6, after the description window. It tells the calculator two things: that the variable exists, and that the value coming next is a real number.

Byte	Field	Value	What it means
1	preamble	`3A`	":"
2–4	command	`56 41 4C`	ASCII "VAL" — a value is being described
5	—	`00`	fixed
6–7	data type	`56 4D`	ASCII "VM" — variable memory
8	—	`00`	fixed
9	status	`01`	01 = variable is in use. 00 = unused

Byte	Field	Value	What it means
10	—	00	fixed
11	status, repeated	01	must equal byte 9
12	variable name	41–5A	the calculator ignores this on a Receive(— it already knows what it asked for. Echo back what arrived in the request anyway
13–19	padding	FF × 7	fill
20–27	literal	56 61 72 69 61 62 6C 65	ASCII "Variable"
28	number kind	52	ASCII "R" = real, and a 16-byte value packet follows
29	terminator	0A	fixed
30–49	padding	FF × 20	fill
50	checksum	273 − name	constant only because every other byte is fixed

Byte 28 is a declaration, and the calculator holds you to it. "R" commits you to sending exactly 16 bytes next. Send a different length and the calculator treats the packet as malformed, transmits 0x22, and displays COM ERROR. This is Grindheim's own worked example of an abort: declaring a real value and then sending the 26 bytes of a complex one. In the typical use case for sending values this byte is always 0x52.

If byte 9 says the variable is unused, no value packet is sent or expected and the transaction ends there. A device does not need this, but it should not be surprised by it.

Worked example — describing variable A

```
3A 56 41 4C 00 56 4D 00 01 00 01 41 FF FF FF FF
FF FF FF 56 61 72 69 61 62 6C 65 52 0A FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF D0

checksum = 273 - 0x41 = 273 - 65 = 208 = 0xD0
```

Variable	Byte 12	Description checksum
N	4E	C3 (195)
T	54	BD (189)
A	41	D0 (208)
B	42	CF (207)
C	43	CE (206)

4.5 Packet 3 — the value (16 bytes, device → calculator)

Sent at step 8, at the end of the value window. This is the one that carries the measurement, and it is the packet worth learning properly.

Byte	Field	Value	What it means
1	preamble	`3A`	":"
2–5	header	`00 01 00 01`	fixed for a single real value
6	integer digit	`01–09`	the digit in front of the decimal point, as a plain number. Normalising guarantees it is 1–9. Only an exact zero puts 00 here
7–13	decimal digits	`BCD`	fourteen digits, two per byte, most significant first
14	sign / information	`00, 01, 50, 51`	§4.5.1
15	exponent	`00–63`	§4.5.2
16	checksum	`varies`	§4.2

Bytes 6 to 15 are the ten-byte number format used throughout this family of calculators: one integer digit, fourteen decimal digits, flags, exponent. Learn those ten bytes and you can read any real value the protocol carries.

4.5.1 Byte 14 — sign and information

A byte of flags. Only three bits are ever set:

Bit	Meaning
7	the value has an imaginary part — complex numbers only, and never used in this manual.
6 and 4	the value is negative. Both bits are set together
0	the magnitude is 1.0 or greater
1, 2, 3, 5	always 0

Which gives just four values you will ever meet:

Byte 14	Meaning	Example
`00`	positive, magnitude below 1	0.5
`01`	positive, magnitude 1 or more	23.5
`50`	negative, magnitude below 1	−0.045
`51`	negative, magnitude 1 or more	−12.75

Bit 0 is not decoration. It works with the exponent byte, and §4.5.2 shows why you cannot read one without the other.

Bits 6 and 4 being set together for a single condition is unexplained in every source examined, and is reported here as observed behaviour rather than as something understood. Encode it as observed: for negative values write 0x50, not 0x40.

4.5.2 Byte 15 — the exponent

The exponent is a plain binary number from 0 to 99. It is not BCD, which catches people out, because every other numeric field in this packet is.

The exponent byte carries no sign of its own. Its sign lives in bit 0 of byte 14 — the same bit that says whether the magnitude is 1 or more, which is exactly the same question asked another way. A negative exponent is stored as 100 minus its magnitude:

Exponent	Byte 15	Bit 0 of byte 14	Arithmetic
+99	63	1	99
+3	03	1	3
+1	01	1	1
0	00	1	0
−1	63	0	100 − 1 = 99
−2	62	0	100 − 2 = 98
−99	01	0	100 − 99 = 1

0x63 means +99 or −1 depending on one bit in the previous byte. This is the single most common decoding mistake in this protocol. Always read byte 14 before you interpret byte 15.

Zero is a special case: the exponent byte is 00 and bit 0 is clear, because the magnitude of zero is below 1.

4.5.3 Worked examples

Every packet below was generated by the encoding rules in this chapter and its checksum verified against §4.2.

23.5 — a typical sensor reading

```
23.5  =  2.35 x 10^1     integer digit 2, decimals 3 5 0 0 0 0 0 0 0 0 0 0 0

3A 00 01 00 01 02 35 00 00 00 00 00 00 01 01 C5
```

Byte	Hex	Field	Reading it back
1	3A	preamble	":"
2–5	00 01 00 01	header	fixed
6	02	integer digit	the 2 in 2.35
7–13	35 00 00 00 00 00 00	decimal digits	3 and 5, then twelve zeros
14	01	sign / information	positive, magnitude 1 or more
15	01	exponent	+1, because bit 0 of byte 14 is set
16	C5	checksum	256 − (59 & 0xFF) = 197

3 — the sensor count, sent for Receive(N)

```
3  =  3.0 x 10^0         integer digit 3, all decimals zero, exponent 0

3A 00 01 00 01 03 00 00 00 00 00 00 00 01 00 FA
```

−0.045 — negative, and below 1

```
-0.045  =  -4.5 x 10^-2  integer digit 4, decimals 5 0 0 ...
                         byte 14 = 0x50  negative, magnitude below 1
                         byte 15 = 100 - 2 = 98 = 0x62

3A 00 01 00 01 04 50 00 00 00 00 00 00 50 62 F8
```

−12.75 — negative, and 1 or more

```
-12.75  =  -1.275 x 10^1   integer digit 1, decimals 2 7 5 0 ...
                           byte 14 = 0x51  negative AND magnitude >= 1

3A 00 01 00 01 01 27 50 00 00 00 00 00 51 01 34
```

1013.2 — atmospheric pressure in hPa

```
1013.2  =  1.0132 x 10^3   integer digit 1, decimals 0 1 3 2 0 ...
                           note byte 7 = 0x01, the digits 0 and 1

3A 00 01 00 01 01 01 32 00 00 00 00 00 01 03 C6
```

0 — a special case

```
every field is zero, so the body sums to 2 and the checksum is
always the same

3A 00 01 00 01 00 00 00 00 00 00 00 00 00 FE
```

4.5.4 Decoding, in the other direction

Given sixteen bytes, recover the number:

```
1.  check the checksum          sum of bytes 2..16, & 0xFF, must be 0
2.  I    = byte 6                                   integer digit
3.  D    = bytes 7..13 unpacked to 14 decimal digits
4.  neg  = (byte 14 & 0x50) != 0
5.  ge1  = (byte 14 & 0x01) != 0
6.  e    = byte 15;  if (!ge1) e = -(100 - e);      unless the value is 0
7.  v    = (I . D) x 10^e ;  if (neg) v = -v
```

4.6 Packet 4 — END (50 bytes, device → calculator)

Sent at step 10. Nothing in it varies.

```
3A 45 4E 44 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF 56

3A          preamble
45 4E 44    ASCII "END"
FF x 45     fill
56          checksum, always
```

Because it is identical every time, most implementations store it as a constant array and transmit it verbatim.

5. Putting it together — what an interval logger actually does

The published calculator program uses five variables. Understanding the sequence of transactions explains why interval logging works with no clock anywhere in the system.

Variable	Direction	Purpose
N	Receive(	how many sensors the device has — asked once, at the start
T	Send(	the sampling interval in seconds — the calculator tells the device
A	Receive(	sensor 1 → List 2
B	Receive(	sensor 2 → List 3, only if N > 1
C	Receive(	sensor 3 → List 4, only if N > 2

The important detail is where the interval wait sits. A, B and C are three separate RECEIVE() transactions, but they are three transactions belonging to one sample. The device waits out the interval in the value window of A only. B and C follow immediately with readings taken at the same moment.

```
if      (vname == 'N')  send_value(SENSOR_COUNT);
else if (vname == 'A') { wait_for_interval();     // <-- here, and only here
                         send_value(sensor[0]); }
else if (vname == 'B')  send_value(sensor[1]);
else if (vname == 'C')  send_value(sensor[2]);
else                    send_value(0);
```

Put wait_for_interval() in all three and your sampling interval silently becomes three times what the student asked for. This is a fault that produces a perfectly smooth graph but with the wrong time axis, which is the hardest kind to notice.

Elapsed time is not transmitted at all. The calculator computes it from what it already knows:

```
  time of reading k  =  T x (k - 1)
```

So no clock is needed at either end. The device counts the interval; the calculator counts the readings. That is the whole trick.

> **Contributed by this work —** One calculator program that never learns which device is attached. The device supplies its own sensor count, so the same Casio program drives a two-channel micro:bit and a three-channel ESP32 without modification. Validated across five microcontroller families and both calculator generations. The two-generation part is carried by the 2007–08 work using the FX-9750G Plus, and 2025 / 2026 work using the GIII.

6. When it does not work

What you see	Most likely cause	What to do
Device waits forever, no error on the calculator	the diode is reversed, or missing and the pull-up is absent	check the diode bar faces the microcontroller; check the pull-up
COM ERROR immediately	the device did not answer 0x15 within 0.5–1 s (Grindheim, 2001)	the device was busy doing something else — see §3.5
Serial monitor reports a short read of 1 byte: 22	the calculator aborted; it sent 0x22 and gave up (Grindheim, 2001)	something the device sent was malformed. Check packet length against byte 28, and check every checksum
Works, then COM ERROR after a few minutes	the device serviced other work between transactions, not inside a window	move that work inside the value window
Works, then COM ERROR after roughly an hour	calculator batteries	fit fresh cells. This ends a session with no software precursor at all
Readings arrive but are nonsense	no pull-up, so the receiver sees a permanent break	fit the 4.7 kΩ pull-up
Graph looks right, time axis is wrong	the interval wait is in more than one of A, B, C	wait in the A branch only — chapter 5

0x22 is Grindheim's finding. It is the calculator saying it received something it could not accept, and it is the most informative failure in the protocol because it tells you the device transmitted; the wiring is sound and the two ends are talking. What is wrong is the content of a packet, which is a much smaller search space than a silent link.

> **Contributed by this work —** Five of the seven rows above. The two exceptions are Grindheim's, and are marked as his in the table: the 0.5–1 second reply timeout, and the 0x22 abort on a malformed packet. Those are the only two failure modes any prior description of this protocol records. The other five exist because the fault was met on the bench, diagnosed and written down; the WiFi service placement in 2026 and the battery-voltage session limit in 2025, the interval-wait placement across the six platform builds, and both wiring faults. None of the five arises when the device on the other end is a personal computer, which is why no prior source could have found them.

7. Free programming software for all platforms

Picaxe Programming Editor

Most hobbyist and commercial users, as well as some educational users, program the PICAXE chip using the easy to learn BASIC language. This language is designed to allow users without any formal programming experience to be able to quickly and simply develop PICAXE microcontroller programs. PICAXE BASIC is much simpler to learn (and to 'debug') than traditional microcontroller languages such as assembler code or 'C'. The software for BASIC programming is completely free and available for Windows, Mac and Linux.

- Windows users should select the PICAXE Editor
- Mac and Linux users should select Visual Studio Code, AXEpad or Blockly
- Chromebook users should use Blockly
- iPad and Android tablet users can use Blockly Cloud in their web browser

https://picaxe.com/software/picaxe/

Arduino IDE - Arduino, ESP8266, ESP32

- Download Arduino IDE
- You will need to use the Desktop IDE. Make sure you are running the latest version.

https://support.arduino.cc/hc/en-us/articles/360019833020-Download-and-install-Arduino-IDE

Install additional boards for each as internet tutorials show.

EXAMPLE: Wemos D1 mini (ESP8266)

To install the Wemos D1 Mini board in the Arduino IDE, add `http://arduino.esp8266.com/stable/package_esp8266com_index.json` to your **Additional Boards Manager URLs** in Preferences, then open the **Boards Manager**, search for ESP8266, and install the package by **ESP8266 Community**.

Add the Board Manager URL

- Open the Arduino IDE.
- Go to **File > Preferences** (or **Arduino IDE > Settings** on macOS).
- Find the field named **Additional Boards Manager URLs**.
- Paste this link into the box: http://arduino.esp8266.com/stable/package_esp8266com_index.json.
- Click **OK**

Install the ESP8266 Package

- Go to **Tools > Board > Boards Manager** (or click the Boards Manager icon on the left sidebar).
- Type ESP8266 into the search bar.
- Find **esp8266** by **ESP8266 Community** and click **Install**.
- Wait for the download and installation to finish

Select Your Wemos D1 Mini

- Connect your Wemos D1 Mini to your computer via a USB cable.
- Go to **Tools > Board**, and look for **ESP8266 Boards**.
- Select **LOLIN(WEMOS) D1 R2 & mini** (or **WeMos D1 R2 & mini**).
- Go to **Tools > Port** and choose the correct COM/Serial port assigned to your device

EXAMPLE: ESP32 WROOM devkit

To install the ESP32 WROOM devkit board manager in the Arduino IDE, open **File > Preferences**, paste `https://espressif.github.io/arduino-esp32/package_esp32_index.json` into **Additional Boards Manager URLs**, open the **Boards Manager**, search for ESP32, and install **esp32 by Espressif Systems**.

Setup Steps

- Open the Arduino IDE software on your computer.
- Go to **File** and click **Preferences** (or *Settings*).
- Find the **Additional Boards Manager URLs** box at the bottom.
- Paste this official link: `https://espressif.github.io/arduino-esp32/package_esp32_index.`
- Click **OK** to save the preferences.

Install the Board Package

- Open the **Boards Manager** by clicking the board icon on the left menu, or go to **Tools > Board > Boards Manager**.
- Type ESP32 in the search text box.
- Find **esp32** made by **Espressif Systems** and click **Install**.
- Wait a few minutes for the download and install process to finish

Select Your WROOM Board

- Plug your ESP32 WROOM devkit into your computer using a USB cable.
- Go to **Tools > Board > esp32** (or search/select) and choose **ESP32 Dev Module** (the standard choice for generic WROOM-32 devkits).
- Go to **Tools > Port** and pick the active COM port assigned to your device.

EXAMPLE: Arduino Uno R3

The "Arduino AVR Boards" package, which contains support for the Arduino Uno R3, comes pre-installed by default in the Arduino IDE. You do not need to manually install a board manager package to use an Uno R3.

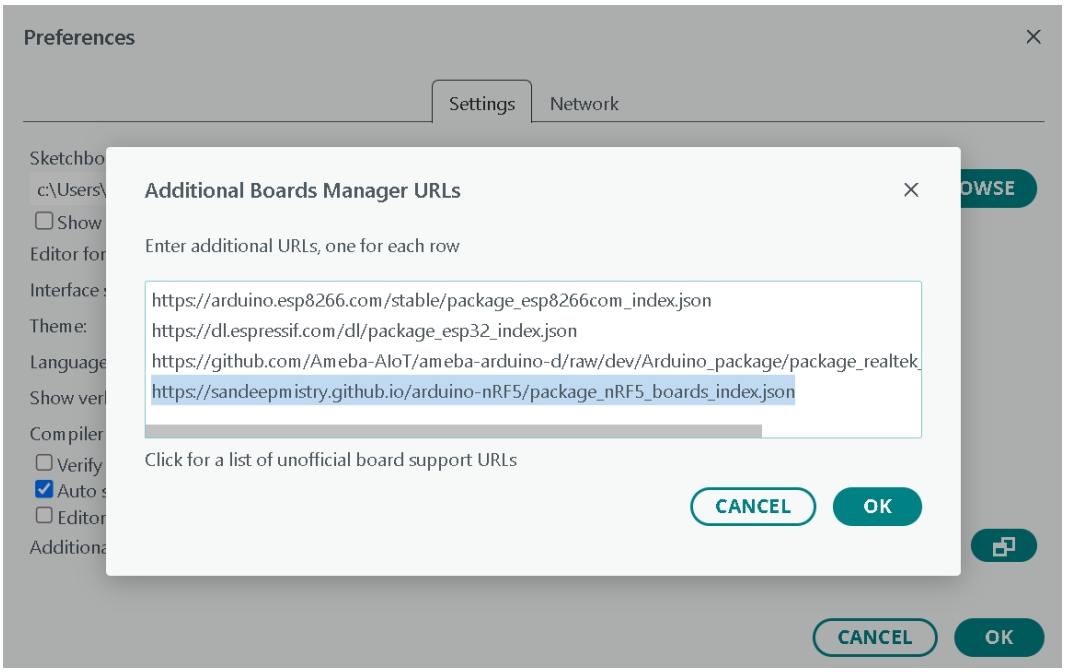
Selecting the Arduino Uno R3

- Plug your Arduino Uno R3 into your computer using a USB cable.
- Open the Arduino IDE.
- Go to the top menu and select **Tools** > **Board** > **Arduino AVR Boards** > **Arduino Uno**.
- Go to **Tools** > **Port** and select the COM port or serial port assigned to your Uno.

Arduino IDE - installing Microbit dependencies

- Download Arduino IDE
- Add NRF5x Board Support. The microbit uses the nRF51 which is not 'natively' supported.

In Arduino, go to Preferences and add *https://sandeepmistry.github.io/arduino-nRF5/package_nRF5_boards_index.json* into the Additional Board Manager URL text box. If this is not your first, make sure to separate URLs with a comma.



Open Tools>Board>Boards Manager from the menu bar, search for nRF5 and install "Nordic Semiconductor nRF5 Boards" by Sandeep Mistry

Github repository is here:

https://github.com/sandeepmistry/arduino-nRF5

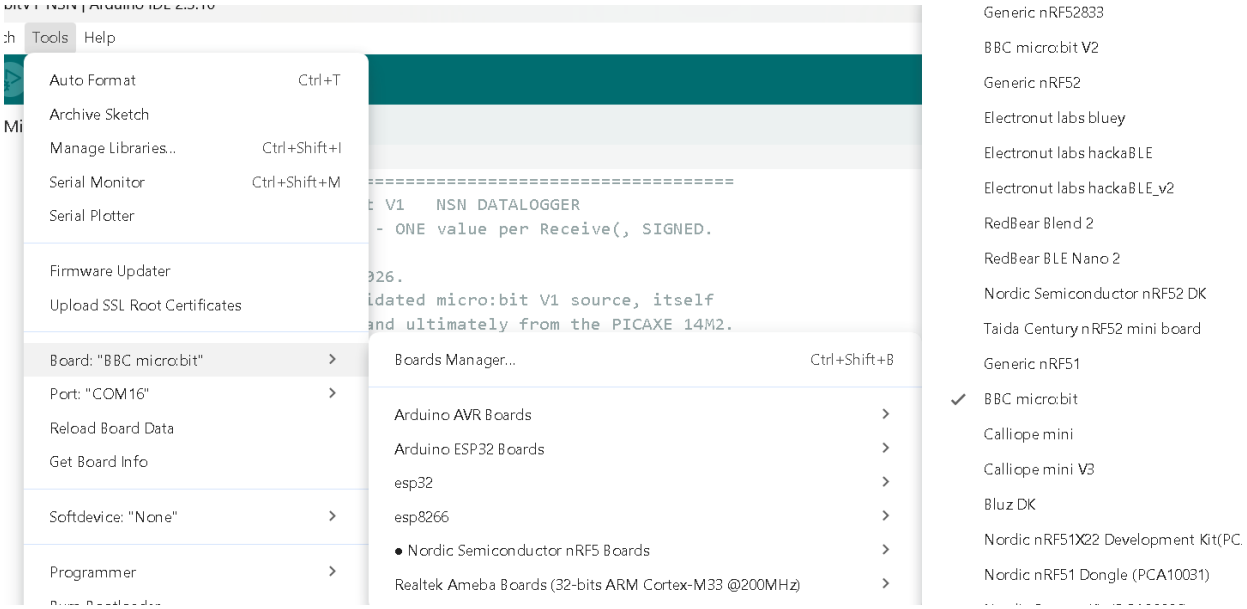
Open the Boards Manager from the Tools -> Board menu and install "Nordic Semiconductor nRF5 Boards"

Select your nRF5 board from the Tools -> Board menu

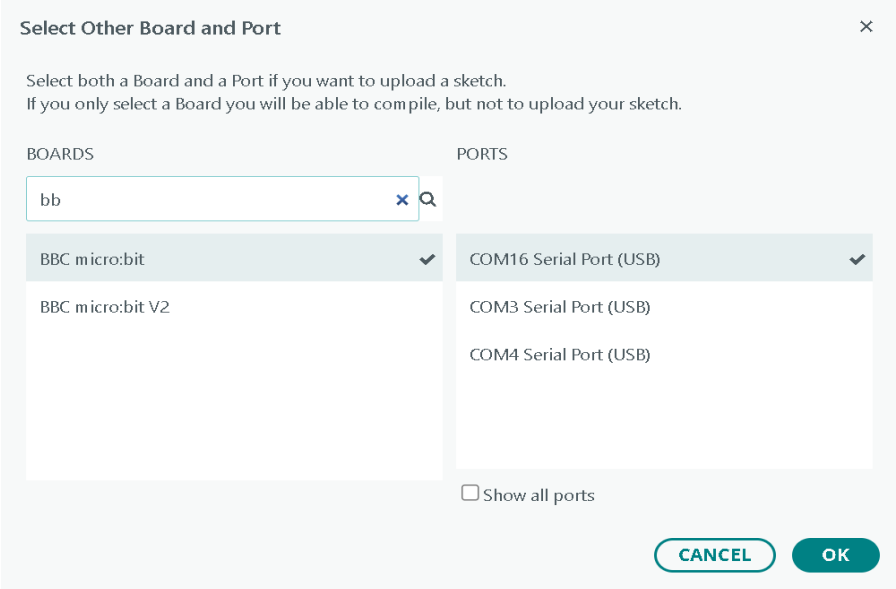
Select BBC micro:bit from the Boards menu:

- Use "BBC micro:bit" for the original BBC micro:bit
- Use "BBC micro:bit V2" for the newer micro:bit

If you select the wrong one, change to the other. The example will not work if you select the wrong board.



And set the Port to the microbit



Create a new sketch with this blink demo

```
const int COL1 = 3;      // Column #1 control
const int LED = 26;      // 'row 1' led

void setup() {
  Serial.begin(9600);

  Serial.println("microbit is ready!");

  // because the LEDs are multiplexed, we must ground the opposite side of the LED
  pinMode(COL1, OUTPUT);
  digitalWrite(COL1, LOW);

  pinMode(LED, OUTPUT);
}

void loop(){
  Serial.println("blink!");

  digitalWrite(LED, HIGH);
  delay(500);
  digitalWrite(LED, LOW);
  delay(500);
}
```

Click Upload!

If you get a warning about openocd - approve access so it can upload the code

8. Project ideas

From the original 2007/2008 research with classroom validated activities

Build it, Test it, Use it – Portable sensor unit

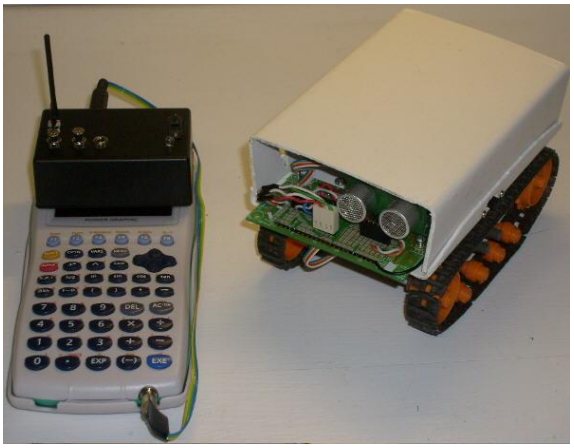
A hand-held portable sensor unit containing the microcontroller, batteries and a 2.5 mm TRS socket for the cross-over cable. The socket is at one side of the unit. Use other 3.5 mm TRS sockets located on the top face for sensors. Plug unused sockets.

The example here is for the Picaxe 18X, using 3x1.5V AAA cells, with a power on/off switch. ALWAYS TURN OFF the unit before inserting or removing sensor plugs!

Other input or output components might include an LED to signal when communication is in progress or errors, and a programming port.

The 3-pin socket provided power and a signal line for a radio transmitter module.

Build it, Test it, Use it – Remote control robot

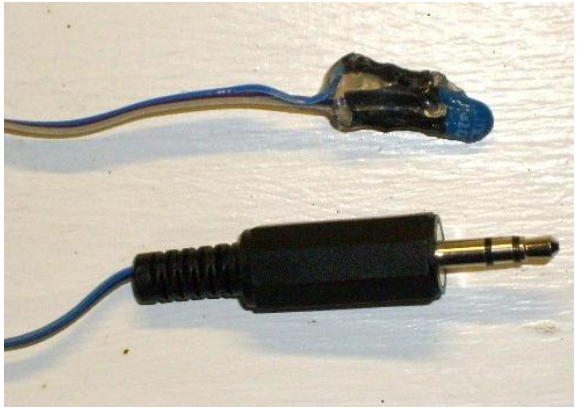


A hand-held portable sensor unit uses the crossover serial cable to connect to the Casio calculator.

The Casio acts as a Human-machine-interface, operating as the keypad with keypresses send wirelessly to a robot. Range was approximately 20 metres.

The 3-pin socket provided power and a signal line for a radio transmitter module.

Build it – a temperature sensor



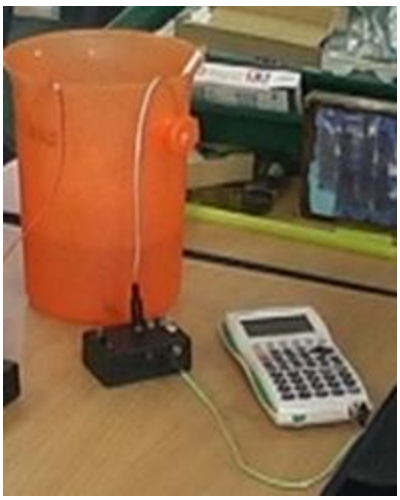
3.5 mm TRS plug connects to sensor unit.

A 10k NTC thermistor has a 10k resistor wired to it at the 3-wire ribbon cable lead connection (see heat shrink region in the photo). It can be in pull-up or pull-down configuration. I use pull-down for junior learners, pull-up for senior classes. It generates good discussions.

NTC thermistor connects to +ve and the fixed resistor connects to ground (acting as a pull-down). The output signal is read by the microcontroller analog input.

As temperature increases, the NTC resistance decreases. This lets more voltage drop across the fixed resistor, and **the output voltage increases.**

Build it, Test it – Calibrate a NTC temperature sensor



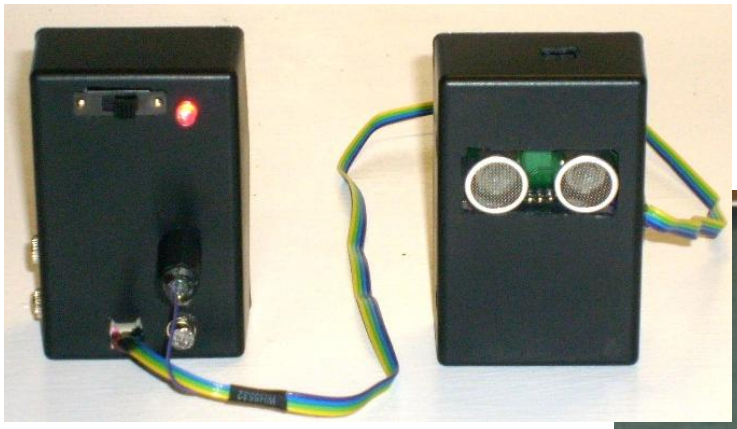
A hand-held portable sensor unit connected to a Casio calculator using the crossover serial cable. The DIY homemade NTC temperature sensor connects to the sensor unit via the 2.5mm TRS plug.

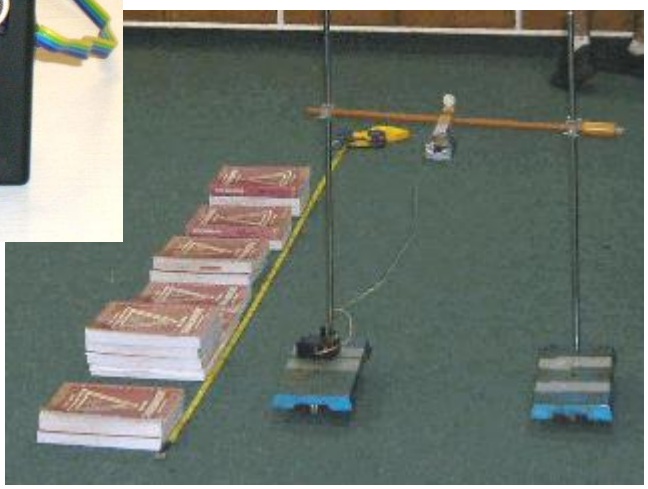
- Ice water, room temperature water, warm/hot 60-70 oC (**NOT BOILING WATER)**
- Manually trigger recordings or use the automatic time interval recording.
- Use the calculator to convert raw analog reading to a temperature using valid equation for NTC thermistors.

Build it, Test it, Use it – An ultrasonic range finder

The HC-SR04 and HC-SR05 ultrasonic range finder modules now come in 5V and 3V versions. The DIY homemade unit shown was wired in 3-pin configuration for power and a signal line. The signal line triggered the module but also received the pulse from the module, permitting the Picaxe to use a single pin to get distance information.

- Sensor unit 3-pin socket provides power and signal to HC-SR04 ultrasonic range finder module.
- Range from 2 cm to approximately 400 cm
- Sensor unit connects to Casio via the crossover serial cable.
- Range finder module can be mounted vertically or horizontally as appropriate.
- Rangefinder and Casio calculator can be mounted in a compact container for aerial “fly over’ mapping of the floor or ground below. Wire aerial guides or a wheeled frame (as used for the Mars Rover mission) have been used.





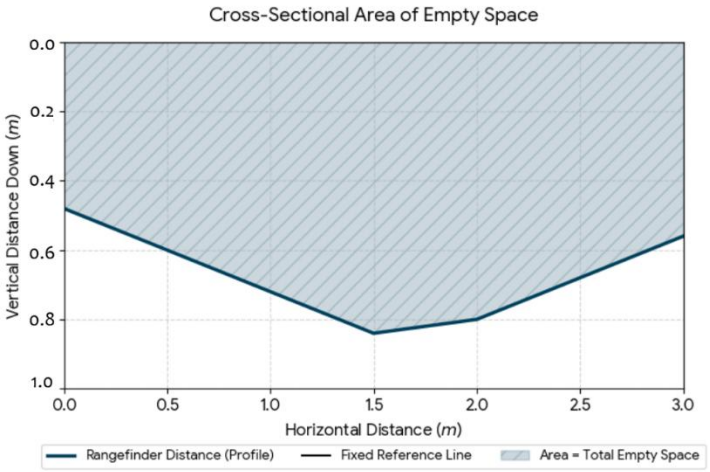
Using a distance sensor / rangefinder

A. Map terrain from the air – Mars Surveyor mission

- **Explore signal loss when slopes are 45 degrees –** simulates limitations of equipment and methods for remote sensing
- **Light level vs distance –** readings from two different sensors for a reflective surface

B. Find area under the curve

In this Mars context, the area under the curve tells the students the total cross-sectional area of the atmosphere (or empty valley) above the ice cap, which is the key first step to calculating the total volume of Martian water ice in that region.



What the Data Tells the "NASA Scientists"

- The Shape of the Ice: Because the sensor is at a fixed point above the ice (like a satellite or a drone), a higher Y-value means the ice is thin (farther from the sensor), and a lower Y-value means the ice is thick (closer to the sensor).
- The "Empty Space" Area: The Area Under the Curve (AUC) calculates the total vertical cross-sectional area of the gap *between* the spacecraft's reference line and the top of the ice cap.
- The Goal (Ice Volume): To find the actual volume of the ice cap, the students will use a subtraction method.

Total Sector Area: Multiply the *satellite orbit height* by the *total horizontal distance scanned*. This gives the area of a perfect rectangle.

Ice Cap Cross-Sectional Area: Subtract the **Area Under the Curve (empty space)** from the **Total Sector Area**. The remaining area is the actual vertical slice of the ice.

Total Ice Volume: Multiply that Ice Cross-Sectional Area by the estimated width of the ice sheet to find the total cubic meters (\(m^{3}\)) of water ice available for a future Mars colony!

Using NTC Temperature Sensors

A. Cooling and Heat-Loss Investigations

- **Cooling curves in different containers.** Compare cooling rates of water in metal, glass, plastic, insulated, or lidded containers.
- **Crime scene investigation.** Create a model curve to solve how long the coffee was left out to check a suspects alibi.
- **Comparing insulation materials.** Wrap identical containers in different materials (paper, cloth, foam) and compare heat loss.
- **Newton's Law of Cooling (Level 2).** Fit exponential curves and extract cooling constants.

B. Temperature Gradients and Heat Transfer

- **Conduction along a metal rod** Thermistors at different distances from a heated end.
- **Convection in a water column** Thermistors at bottom/middle/top to observe heating and mixing.
- **Temperature stratification in air or water** Measure vertical gradients in a room, greenhouse, or tank.

C. Sensor Response and Calibration

- **Response time of thermistors** Compare bare vs insulated vs mass-loaded sensors during sudden temperature changes.
- **Calibration and linearisation** Use ice water, room temp, warm water, hot water to build calibration curves.
- **Comparing three "identical" thermistors** Explore manufacturing tolerances and measurement uncertainty.

D. Energy and Mixing

- **Mixing hot and cold water** Measure initial temps and final mixture temp; compare to energy-conservation prediction.

Using Light Sensors

A. Light Intensity and Distance

- **Inverse square law (Level 2)** Photodiode/phototransistor or LDR measuring intensity vs distance.
- **Light level vs distance (Level 1)** Simple pattern-finding with an LDR.

B. Shadows, Reflection, and Filters

- **Shadow size vs object distance** LDR behind an object moved closer/further from a lamp.
- **Reflectivity of surfaces** Compare white/black/foil/coloured/rough/smooth surfaces.
- **Colour filter transmission** Measure how different coloured filters affect transmitted light.

C. Spectral Sensitivity

- **UV vs visible vs IR response** Use UV LED (reverse-biased), IR LED (reverse-biased), and photodiode to compare responses to:
  - Sunlight
  - Indoor lighting
  - IR remote
  - UV lamp (safe source)

D. Optical Communication

- **IR LED transmitter + IR detector** Measure signal strength vs distance and alignment.

E. Building a Simple Lux Meter

- **Sensor linearity and calibration** Compare sensor output to estimated intensity (distance) or a commercial lux meter.

Biology Investigations (Using Light Absorption / Transmittance)

A. Enzyme Reactions

- **Amylase breaking down starch (iodine fading)** Track colour fading over time using transmitted light.
- **Catalase/peroxidase reactions with coloured indicators** Monitor colour change to measure reaction rate.

B. Photosynthesis

- **DCPIP decolourisation** Measure rate of DCPIP reduction under different light intensities or distances.

C. Turbidity and Growth

- **Yeast growth.** Track increasing turbidity (decreasing transmission) over time.
- **Safer alternative:** yeast + sugar fermentation.

D. Light-dependent biological processes

- **Effect of light colour on photosynthesis** Use coloured filters and measure DCPIP reduction rate.

Chemistry Investigations (Using Light Absorption / Transmittance)

A. Beer–Lambert Law

- **Food dye concentration vs absorbance** Make serial dilutions and measure transmitted light; plot absorbance vs concentration.

B. Reaction Kinetics via Colour Change

- **Permanganate reduction** Track fading purple colour over time.
- **Iron(III) + thiocyanate complex formation** Track red colour formation.

C. pH Indicators

- **Indicator colour vs pH** Measure transmitted light through solutions of known pH.
- **Indicator behaviour during titration** Log light level continuously as pH changes.

D. Precipitation and Turbidity

- **Formation of insoluble salts** Track decreasing transmission as precipitate forms.
- **Rate comparison** Vary concentration or temperature.

Curriculum integration – sample only – excludes new industry-led subjects

Physics:

- Thermodynamics (heat capacity, cooling laws)
- Optics (light intensity measurements, reflectance, absorbance, transmission)
- Mechanics (motion sensors with ultrasonic)
- Ion chamber – alpha particle detection
- Expansion rates
- Sound / noise level surveys
- Sound proofing techniques
- Static electric fields
- Magnetic fields
- Ultrasonic range finding

Chemistry:

- Kinetics (reaction rate monitoring)
- Thermochemistry (exothermic / endothermic reactions)
- Colorimetry (colour change and rates of reaction)
- Electrochemistry (voltage/current logging)

Biology:

- Photosynthesis (CO_2 and light)
- Respiration rates (O_2 consumption)
- Health sciences (breathing rates , heart rates)
- Bioluminescence

Environmental / Horticultural Science:

- Atmospheric monitoring
- Soil moisture monitoring and water quality testing
- Sound / noise level monitoring
- Home environment monitoring / sampling
- Pasture / crop conditions sampling / monitoring

Mathematics and Statistics:

- Graphing, curve fitting, central tendency (mean), spread (standard deviation)

Calculus:

- Differentiation (rates of change) / Integration (areas from terrain mapping using a range finder) / Cooling curves.

Digital Technology / Computer Studies / Robotics and Automation

- Applications of binary code and hexadecimal
- Wireless telemetry and remote control
- Measurement and Control systems
- Coding languages for microcontrollers
- GPS and navigation

Physical education:

- Warming up and cooling down
- Heart rate and respiration recovery times

Space technology:

- Wireless telemetry and remote control – Voyager example, GPS and navigation.
- Robotics and automation – Mars Rover example
- Satellite surveying – Mission to Mars example
- Measurement and Control systems – new lunar missions; Space X and NASA
- Coding languages for microcontrollers – maintaining legacy systems – Voyager example

Music / Science Communication / Digital media creation

- Create songs and video clips to promote Science, Mathematics, Engineering, Coding and Robotics

https://www.youtube.com/@nznrg

The robot B9 from the TV show “Lost in Space” (4 versions for different school age groups)

https://youtu.be/Dm2rruRP2XI - Roll with me B9 (the nature of science and seeing first-hand)

https://youtu.be/yCJ5pwdDt6A

https://youtu.be/0Cg1Fd_HSMU

Coding and AI as a partner – soundtracks to make learning feel good!

https://youtu.be/UJegjTEAs5A - Apple II Transylvania remake for modern web browsers

Props and models – perspective and engineering in science fiction – music track for “Land of the Giants” TV

https://youtu.be/s1xazris0Pk - Ballad of the Spindrift

9. Exercises

Answers can be checked with the rules in chapter 4 and a calculator. Nothing here needs hardware.

Reading

1. What number is this value packet? 3A 00 01 00 01 07 25 00 00 00 00 00 00 00 62 70
2. Verify its checksum by the receiver's method: add every byte after the preamble, including the checksum, and mask to eight bits.
3. A packet arrives with byte 14 = 0x51 and byte 15 = 0x63. What is the exponent, and how do you know?

Writing

4. Write the full 16-byte value packet for 100.0.
5. Write it for −0.0072.
6. Write the request packet the calculator sends for Receive(B), and compute its checksum. Check it against the table in §4.3.

Reasoning

7. A student's logger works on the bench but raises COM ERROR once they add a small OLED display that is refreshed after each reading. Where is the refresh happening, and where should it happen?
8. Why can the same cable serve a 2.75 V calculator and a 4.75 V one without any change? Answer in terms of what the microcontroller is and is not allowed to do to the line.
9. A device sends a description packet with byte 28 = 0x52 and then transmits 26 bytes. What does the calculator do, and which byte does it send?
10. Explain why a wrong checksum is more useful to a device than a right one that happened by luck.

10. Sources, and what is contributed here

10.1 Prior work

- **Erik Grindheim (2001)** — Casio CFX-9950G communications protocol, revision 1.2, 7 October 2001. The packet structures, the number format, the checksum, the 0.5–1 second reply deadline, the 0x22 abort and the electrical description all originate here. Written from a personal computer, which explains what it does not contain: no turnaround delay, no host-wait window, and no question about when a device may do other work.

  Every byte table in chapter 4 was checked against that document line by line, and Grindheim's own worked checksum for the END packet — sum 0xE4, less 0x3A, invert, add one, giving 0x56 — was reproduced and agrees with the shorter rule stated in §4.2. Where this guide states something he does not, it is marked. Where it restates him, it is his.

- **Tom Lynn (1999)** — earlier community documentation of the interface.
- **Michael Fenton (2004–2008)** — adapted Grindheim's work after correspondence with Andrew Hornblow, and published PICAXE-to-Casio example code and protocol notes on the Revolution Education forum. Published first edition "Casio Picaxe Manual" 2008, referred to by Anobium and nsg21.
- **Anobium (2012) and nsg21 (2018)** — both built on that published code, and both credited it.
- **MiniExperimenter (shabaz123, 2020)** — the most technically sophisticated independent prior work identified. It uses the EA-200 hardware protocol at a different baud rate, and does not address interval timing.

10.2 What this document contributes

Collected from the callouts throughout, in the order they appear:

Section	Contribution	When
§2.2	The pull-up is required to define an idle state, not to compensate for weak drive. Measured behaviour of the port when not in use: 0 V at about 120 kΩ	October 2025
§2.3	The diode as an open-drain translator on the SB-62 port, and the demonstration that the calculator's receive line has its own internal pull-up. Documented DOI 10.5281/zenodo.22004530	2008 first edition of this manual, revalidated Jan 2026
§3.1	That two stop bits to the calculator are also accepted, which Grindheim does not state. An earlier claim by this project — that one is rejected — withdrawn	Apr 2026
§3.4	The four host-wait windows, found by exhaustive search, including the positions where a pause fails	2007 and Oct 2025
§3.4	Precise interval logging — holding the calculator in the value window while the device keeps time	Oct 2025
§3.4	The 300 s specified range, the three-hour demonstration. Note: the withdrawal of an earlier maximum shown to be a battery-voltage artefact	October 2025
§3.5–3.6	Where a device may safely do other work, and why a microcontroller meets constraints a PC never does	Oct 2025
§4.2	Verification of Grindheim's checksum rule against every independently known constant. The short restatement is presentation, not a finding	Jan 2026
Chapter 5	One calculator program serving five microcontroller families, with the device declaring its own sensor count	Oct 2025 – Apr 2026
Chapter 6	Six of the eight diagnostic entries — bench faults met, diagnosed and recorded. The other two are Grindheim's	2007–2026

The byte-level structures in chapter 4 are Grindheim's, restated for a microcontroller reader with worked examples generated and verified afresh. Where this document differs from prior descriptions, the difference is noted at the point it occurs.

10.3 Not claimed

Using a diode as an open-drain level translator is standard practice in mixed-voltage interfacing and is not claimed as novel. What is recorded is its application to this particular interface, arrived at independently in 2007, and appears to be the earliest known use of the configuration on the Casio SB-62 port.

10.4 Deliberately withheld

Two results are omitted from this guide entirely:

- A method of encoding several synchronous sensor readings into a single RECEIVE() transmission. The method, its field structure and its decoder are withheld pending peer-reviewed publication.
- A previously undocumented condition under which the calculator's own number handling silently loses or corrupts part of a received value. It is deterministic and reproducible, it is a property of the calculator rather than of anything in this project, and it does not affect the method described here. The condition and the design rule that avoids it are withheld for the same reason.

Neither omission leaves a gap in what this guide describes.

10.5 Citing this work

Fenton, M. (2026). RIGEL Casio Serial Protocol Technical Manual. Zenodo. https://doi.org/10.5281/zenodo.22003135

AND

Fenton, M. (2026). Casio Graphing Calculator Serial Interface: Priority Disclosure of Timing Discoveries, Encoding Invention, and Operational Modes (FX-9750 and FX-9860 Series). Zenodo. https://doi.org/10.5281/zenodo.19303911

Related:

- Fenton, M. (2008). Authentic Learning Using Mobile Sensor Technology. New Zealand Ministry of Education E-Learning Fellowship report. Zenodo. https://doi.org/10.5281/zenodo.19302276
- Fenton, M. (2009). RIGEL — Learning From Life: Communities of Learning via a Connected Curriculum. Microsoft Partners in Learning Regional Innovative Teachers Conference, Kuala Lumpur. Zenodo. https://doi.org/10.5281/zenodo.19334228

Appendix A: Casio BASIC quick reference guide

- Use the menu to navigate to the Program (PRGM) icon and press EXE. Then press F3 to start a new program.
- Alternatively, select a program from the program list and press the F2 (Edit) key to open it for modifications.
- To store a value into a variable you must use the → symbol, which is above the AC button on a FX-9750. Other symbols are found symbols are under the CHAR F6 button. (SYM on older models). You may need to EXIT back up the menu tree to find this, then press SHIFT + VAR again to find the program editor options below.
- If you get back to the top of the menu tree, you should see F1 LIST which is required when programming and F3 DIM.
- Back at the top of the tree press F6, then look for F4 NUM to access the Int (integer), Frac (fraction) and Rnd (round) if needed in your program calculations.

The **Locate** command:

- Locate is used in text based games, it allows you to place text anywhere on the screen using Y, X coordinates. NOTE: Not X, Y
- Locate does not cause the screen to scroll since you cannot place anything below the 7th line. If you try to do this an error is generated. Locate does not wrap onto the next line.
- Take care not to exceed the boundaries of the screen, which is 7 rows high by 21 characters wide.
- Syntax for locate: Locate Y, X, ”YOUR TEXT IN QUOTES HERE” Placing anything outside these boundaries will result in an error.

The **Text** command:

- Text places the text on the graph screen (e.g., next to that a parabola).
- Normal text only screen and the graph screen are independent of each other. You can switch back and forth between the two which can be useful in games.
- Syntax for Text: Text Y, X, ”YOUR TEXT IN QUOTES HERE”

Note: When using Text the Y value comes first and the X value second. You cannot exceed the boundaries of the screen with Text which uses pixel coordinates, 63 high by 127 wide.

ClrText - Clears all the text on the text screen.

Cls - Clears the entire graph screen.

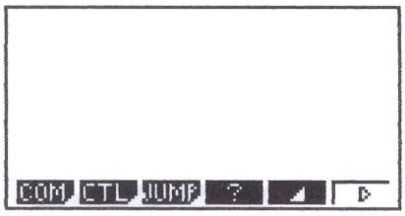
ClrGraph - Clears the graph and sets the view window (covered later) to its default.

Casio calculator keypad key codes

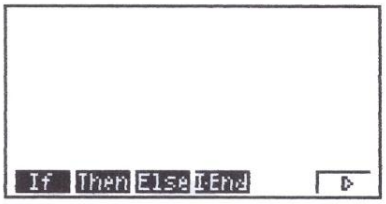
Key Label	Key Code
0	71
1	72
2	73
3	74
4	75
5	76
6	77
7	78
8	79
9	80
. (decimal)	81
EXP	82
(-)	83
AC	30
DEL	31
SHIFT	44
ALPHA	45
MODE	46
X,θ,T	47
EXIT	48
SHIFT+EXIT (QUIT)	49
F1	91
F2	92
F3	93
F4	94
F5	95
F6	96

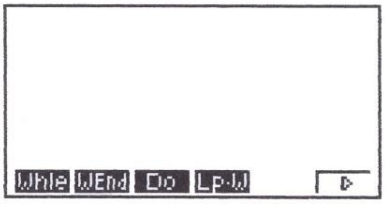
The Programming PRGM Tool Menu

SHIFT + VAR

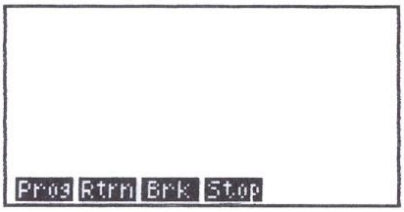


COM menu

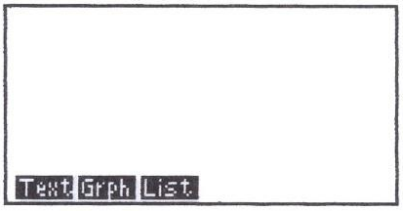




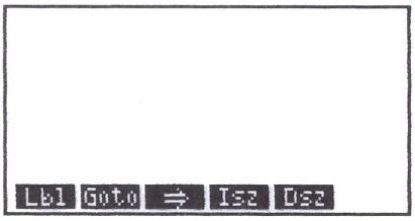
CTL menu



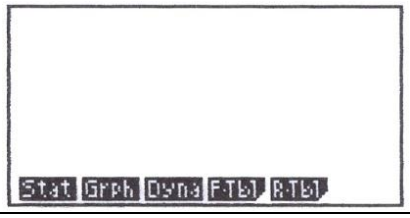
CLR menu



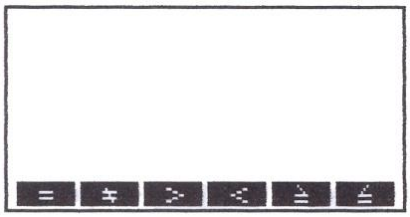
JMP



DISP



REL



I/O

Author: Michael Fenton MRSNZ

Related Zenodo records:

- Authentic Learning Using Mobile Sensor Technology (2008): https://doi.org/10.5281/zenodo.19302276
- RIGEL — Learning From Life, Kuala Lumpur (2009): https://doi.org/10.5281/zenodo.19334228
- Casio Calculator Data Logger Technical Reference: https://doi.org/10.5281/zenodo.19281514